

parallel dfs explanation

13 May 2026 01:42

HPC Assignment 2 — Parallel DFS using OpenMP

This assignment is:

Design and implement Parallel Depth First Search (DFS) using OpenMP

using:

- graph traversal
- stack
- OpenMP synchronization

1. What We Have To Do?

We have to:

1. create graph using adjacency matrix
2. traverse graph using DFS
3. apply parallel programming concepts using OpenMP
4. safely manage shared stack access

Main Goal

To perform:

Depth First Search traversal

while introducing:

parallelism and synchronization

2. What is DFS?

DFS =

Depth First Search

Graph traversal algorithm that:

goes as deep as possible first

before backtracking.

Example

Graph:

```
  0
 / \
1   2
/\
3   4
```

DFS from 0:

0 1 3 4 2

Why Called “Depth First”?

Because:

- traversal first goes deep into one branch

- then returns back

3. Difference Between BFS and DFS

BFS	DFS
level by level	depth wise
queue used	stack used
broader traversal	deeper traversal

4. Why Stack Used in DFS?

DFS follows:

LIFO (Last In First Out)

So stack is ideal.

Example

Push:

1 2 3

Top becomes:

3

Pop order:

3 → 2 → 1

5. Overall Workflow

Input Graph



Push Start Node



Pop Top Node



Visit Node



Push Adjacent Nodes



Repeat Until Stack Empty

6. Code Explanation Line by Line

Header Files

```
#include <iostream>
```

```
#include <vector>
```

```
#include <stack>
```

```
#include <omp.h>
```

Purpose

Header	Purpose
iostream	input/output
vector	adjacency matrix

stack DFS stack

omp.h OpenMP

Namespace

using namespace std;

Main Function

int main()

Execution starts here.

Number of Vertices

int n;

cout << "Enter number of vertices: ";

cin >> n;

User inputs total graph nodes.

Adjacency Matrix

vector<vector<int>>> graph(n, vector<int>(n));

Creates:

graph matrix

Input Graph

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n; j++) {  
        cin >> graph[i][j];  
    }  
}
```

Reads adjacency matrix.

Starting Vertex

int start;

cout << "Enter starting vertex: ";

cin >> start;

DFS traversal starting point.

Visited Array

vector<bool> visited(n, false);

Tracks:

visited nodes

Prevents revisiting.

DFS Stack

stack<int> st;

Stores nodes for DFS traversal.

Push Start Node

st.push(start);

Adds starting node into stack.

Output Message

```
cout << "DFS Traversal: ";
```

Main DFS Loop

```
while (!st.empty())
```

Loop continues until:

stack becomes empty

Current Node Variable

```
int node;
```

Stores current processing node.

Critical Section

```
#pragma omp critical
```

MOST IMPORTANT.

Ensures:

only one thread accesses stack at a time

Why Needed?

Stack is:

shared resource

Multiple threads simultaneously doing:

- pop()
- push()

can corrupt stack.

Safe Stack Pop

```
node = st.top();
```

```
st.pop();
```

Removes top node safely.

Check Visited

```
if (!visited[node])
```

Ensures node processed only once.

Mark Visited

```
visited[node] = true;
```

Node marked as explored.

Print Node

```
cout << node << " ";
```

Displays DFS traversal order.

Traverse Neighbors

```
for (int i = n - 1; i >= 0; i--)  
Checks adjacent nodes in reverse order.
```

Why Reverse Order?

Because stack is:

LIFO

Reverse traversal maintains proper DFS order.

Example

Suppose neighbors:

1 2 3

Push reverse:

3 2 1

Stack pops:

1 first

Correct DFS behavior maintained.

Edge Check

```
if (graph[node][i] == 1 && !visited[i])
```

Means:

- edge exists
- neighbor unvisited

Push Neighbor

```
st.push(i);
```

Neighbor added for future traversal.

End Program

```
return 0;
```

7. Example Execution

Graph:

```
  0  
 / \  
1   2  
/\  
3   4
```

Adjacency matrix:

```
0 1 1 0 0  
1 0 0 1 1  
1 0 0 0 0  
0 1 0 0 0  
0 1 0 0 0
```

Start node:

0

Iteration 1

Stack:

[0]

Pop:

0
Push:
2 1

Iteration 2

Stack:
[2,1]
Pop:
1
Push:
4 3

DFS Traversal

0 1 3 4 2

8. Parallelism in This Program

This assignment mainly demonstrates:

synchronization using OpenMP

Not full DFS parallelization.

Why DFS Difficult to Parallelize?

DFS is:

inherently sequential

because:

- next node depends on previous traversal path

OpenMP Used Here For

Safe access to:

shared stack

using:

```
#pragma omp critical
```

9. Race Condition

Without critical section:

Two threads may:

- pop same node
- corrupt stack
- cause crash

Critical Section Prevents

simultaneous stack modification

10. Time Complexity

Sequential DFS:

$O(V+E)$

Where:

- (V) = vertices
- (E) = edges

Parallel DFS Complexity

Approximate theoretical:

$O\left(\frac{V+E}{P}\right)$

But DFS gains less speedup compared to BFS because:

- traversal path sequential
- synchronization overhead higher

11. Compilation Command

```
g++ -fopenmp dfs.cpp -o dfs
```

Run

```
./dfs
```

Why -fopenmp Needed?

Enables:

OpenMP compiler support

12. Important Viva Questions

Q1. What is DFS?

Depth First Search traverses graph deeply before backtracking.

Q2. Why stack used in DFS?

Because DFS follows LIFO order.

Q3. What is adjacency matrix?

2D graph representation showing connections between vertices.

Q4. Why visited array used?

To avoid revisiting nodes.

Q5. What does #pragma omp critical do?

Allows only one thread at a time to access critical code section.

Q6. Why critical section needed here?

To safely access shared stack.

Q7. What is race condition?

Incorrect behavior caused by simultaneous shared data access.

Q8. Why DFS harder to parallelize than BFS?

Because DFS traversal path is sequentially dependent.

13. Short Practical Explanation

This program implements Depth First Search using a stack and adjacency matrix representation of graph. OpenMP critical sections are used to safely manage shared stack

operations during parallel execution and prevent race conditions.